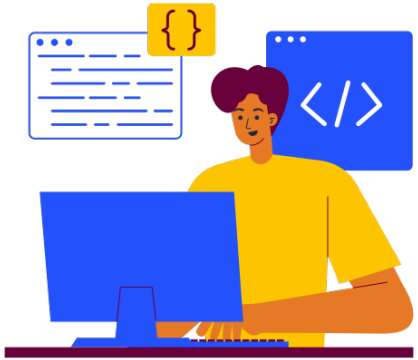
NANYANG THE INNOVATIVE POLYTECHNIC School of Information Technology

SINGAPORE INSTITUTE OF TECHNOLOGY

Full Stack Secured Development

Sequelize ORM and Input Validation



COPYRIGHT © 2024. SINGAPORE INSTITUTE OF TECHNOLOGY AND NANYANG POLYTECHNIC. ALL RIGHTS RESERVED.

Learning Outcomes

By the end of this lesson, you will be able to:

1. Describe Sequelize ORM.
2. Understand Sequelize CRUD Operations.
3. Describe Sequelize Associations.
4. Implement Input Validation using Yup.

COPYRIGHT © 2024. SINGAPORE INSTITUTE OF TECHNOLOGY AND NANYANG POLYTECHNIC. ALL RIGHTS RESERVED.

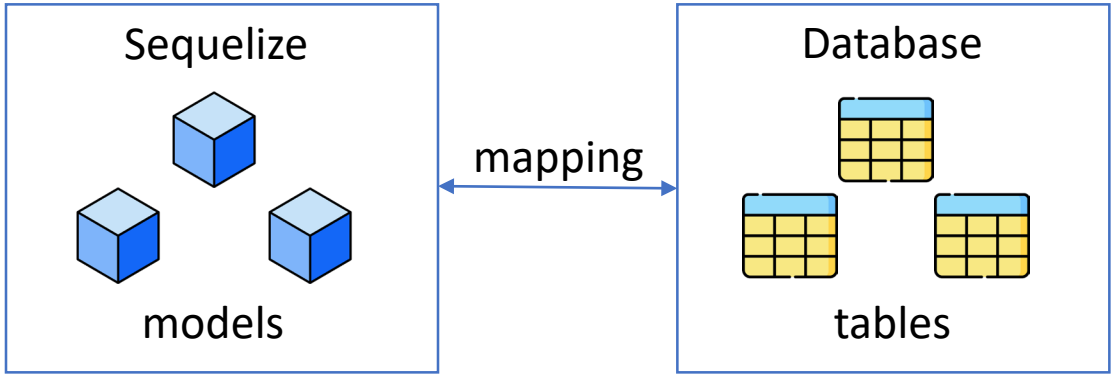
Sequelize ORM

COPYRIGHT © 2024. SINGAPORE INSTITUTE OF TECHNOLOGY AND NANYANG POLYTECHNIC. ALL RIGHTS RESERVED.

Sequelize ORM

Sequelize v6 used

- **Sequelize** is an Object-Relational Mapping (ORM) library for Node.js that provides a layer between your application and the database.
- It allows you to work with objects and perform database operations using JavaScript methods, rather than writing raw SQL queries.



COPYRIGHT © 2024. SINGAPORE INSTITUTE OF TECHNOLOGY AND NANYANG POLYTECHNIC. ALL RIGHTS RESERVED.

Key Features

Here are some key features of Sequelize:

- **Database Agnostic**: supports multiple relational databases, including MySQL, PostgreSQL and SQLite
- **Model-Driven**: allows you to define models that represent database tables
- **CRUD Operations**: provides intuitive methods for creating, reading, updating, and deleting records in the database
- **Association Management**: allows you to define relationships between models using associations, such as one-to-many relationship

COPYRIGHT © 2024. SINGAPORE INSTITUTE OF TECHNOLOGY AND NANYANG POLYTECHNIC. ALL RIGHTS RESERVED.

Installing

Open terminal or command prompt

```
> npm i sequelize
```

- Install sequelize

Install the driver for your database of choice:

```
> npm i mysql2          # MySQL
```

- Known for its speed, reliability, and ease of use

```
> npm i pg pg-hstore    # PostgreSQL
```

- Known for its advanced features and extensibility

```
> npm i sqlite3         # SQLite
```

- Lightweight local database

COPYRIGHT © 2024. SINGAPORE INSTITUTE OF TECHNOLOGY AND NANYANG POLYTECHNIC. ALL RIGHTS RESERVED.

Sequelize Instance

To connect to the database, you must create a Sequelize instance.

```
// Create sequelize instance for MySQL
let sequelize = new Sequelize(
  process.env.DB_NAME,
  process.env.DB_USER,
  process.env.DB_PWD,
  {
      host: process.env.DB_HOST,
      port: process.env.DB_PORT,
      dialect: 'mysql',
      logging: false,
      timezone: '+08:00'
  }
);
```

- The code uses process.env to access **environment variables** for sensitive configuration like the database name, username, password, host, and port.
- **logging: false**: This option disables logging of SQL queries to the console. If set to true, Sequelize would log all executed queries.
- **timezone: '+08:00'**: This option sets the timezone for the database connection. In this case, it's set to UTC+08:00 for Singapore local time.

COPYRIGHT © 2024. SINGAPORE INSTITUTE OF TECHNOLOGY AND NANYANG POLYTECHNIC. ALL RIGHTS RESERVED.

Sequelize Model

Sequelize models are used for defining the structure of the database table, specifying its columns and data types, and defining associations with other Sequelize models.

```
module.exports = (sequelize, DataTypes) => {
    const Product = sequelize.define("Product", {
        name: {
            type: DataTypes.STRING,
            allowNull: false,
        },
        price: {
            type: DataTypes.DECIMAL(10, 2),
            allowNull: false,
        }
    }, {
        tableName: 'products'
    });

    return Product;
}
```

- Model name: **Product**
- Attribute name is using **string** data type
- Attribute price is using **decimal** number with 10 digits and 2 decimal places
- Table name: **products**
- Sequelize will automatically generate the **id** field as the primary key, with integer type.

COPYRIGHT © 2024. SINGAPORE INSTITUTE OF TECHNOLOGY AND NANYANG POLYTECHNIC. ALL RIGHTS RESERVED.

Data Types

Commonly used data types in Sequelize:

- **STRING**: Represents a variable-length string. You can specify the maximum length using STRING(length), e.g. STRING(50).
- **TEXT**: Represents a longer text without a specific length limit.
- **INTEGER**: Represents an integer number.
- **DECIMAL**: Represents a decimal number. You can specify the total number of digits and the number of decimal places, e.g. DECIMAL(10, 2) for price.
- **BOOLEAN**: Represents a boolean value (true or false).
- **DATE**: Represents a date with time.
- **DATEONLY**: Represents a date without time.

COPYRIGHT © 2024. SINGAPORE INSTITUTE OF TECHNOLOGY AND NANYANG POLYTECHNIC. ALL RIGHTS RESERVED.

Synchronizing Models

sync is a Sequelize method that synchronizes the models with the database schema.

```
sequelize.sync({ alter: true })
    .then(() => {
        // start your server here
    })
    .catch((err) => {
        console.log(err);
    });
```

- Setting **alter** to **true** tells Sequelize to make any necessary changes to the database schema to match the model definitions.

- **Synchronize** is commonly used during the project **development** to automatically make any necessary changes to the database structure without requiring manual intervention.
- However, for **production**, it's recommended to use database **migrations** for more controlled schema changes, to prevent data loss or unexpected consequences.

COPYRIGHT © 2024. SINGAPORE INSTITUTE OF TECHNOLOGY AND NANYANG POLYTECHNIC. ALL RIGHTS RESERVED.

CRUD Operations

COPYRIGHT © 2024. SINGAPORE INSTITUTE OF TECHNOLOGY AND NANYANG POLYTECHNIC. ALL RIGHTS RESERVED.

Create

endpoint ‘POST /product’: create a new product

routes/product.js

```
const { Product } = require('../models');

// Define a POST endpoint to create a new product
router.post("/", async (req, res) => {
    let data = req.body;
    let result = await Product.create(data);
    res.json(result);
});
```

- Import sequelize model
- Use the Product model **create()** method to insert one row into the database table using the data received in the request body

Asynchronous Functions (with await)

- Sequelize provides **asynchronous** methods that return Promises. Using **await** with these methods allows you to write non-blocking code, which is the preferred approach in Node.js applications.

COPYRIGHT © 2024. SINGAPORE INSTITUTE OF TECHNOLOGY AND NANYANG POLYTECHNIC. ALL RIGHTS RESERVED.

Read (find all)

endpoint ‘GET /product’: list all products

routes/product.js

```
const { Op } = require('sequelize');

// Define a GET endpoint to list all products
router.get('/', async (req, res) => {
    let minPrice = parseFloat(req.query.minPrice) || 0;
    let list = await Product.findAll({
        // Options for filtering and ordering results
        where: {
            price: { [Op.gte]: minPrice }
        },
        order: [['price', 'ASC']],
    });
    res.json(list);
});
```

- Read minimum price from query parameters
- The method **findAll()** returns an array of model instances, where each instance corresponds to a row in the database table.
- Option **where**: Filter products with a price greater than or equal to the minimum value
- Option **order**: Order the results by the 'price' column in ascending order

COPYRIGHT © 2024. SINGAPORE INSTITUTE OF TECHNOLOGY AND NANYANG POLYTECHNIC. ALL RIGHTS RESERVED.

Sequelize Operators

Sequelize provides a range of operators that you can use to define conditions in your database queries. They are available in the **Op** object.

Implicit Operators

```
brand: null
```

- **Op.is**: brand is null

```
brand: 'Apple'
```

- **Op.eq**: brand equals ‘Apple’

```
brand: ['Apple', 'Samsung']
```

- **Op.in**: brand in the array with ‘Apple’, ‘Samsung’

COPYRIGHT © 2024. SINGAPORE INSTITUTE OF TECHNOLOGY AND NANYANG POLYTECHNIC. ALL RIGHTS RESERVED.

Sequelize Operators

Comparison Operators

- Op.gt: Greater than
- Op.gte: Greater than or equal to
- Op.lt: Less than
- Op.lte: Less than or equal to

```
price: { [Op.gte]: 50 }
```

Between Operator

- Op.between: Between two values

```
price: { [Op.between]: [50, 100] }
```

Like Operator

- Op.like: Matches a string pattern

```
name: { [Op.like]: '%iPhone%' }
```

COPYRIGHT © 2024. SINGAPORE INSTITUTE OF TECHNOLOGY AND NANYANG POLYTECHNIC. ALL RIGHTS RESERVED.

Logical Operators

AND

- A single object can specify multiple conditions to check.
- If you do not specify any combination operator, Sequelize will default to joining them with **Op.and** operator.

```
{
    category: 'Electronics',
    price: { [Op.gt]: 100 }
}
```

- Implicit **Op.and**: category equals ‘Electronics’ **and** price greater than 100

COPYRIGHT © 2024. SINGAPORE INSTITUTE OF TECHNOLOGY AND NANYANG POLYTECHNIC. ALL RIGHTS RESERVED.

Logical Operators

OR

- To replace that implicit AND with an OR, you can use the **Op.or** operator.

```
[Op.or]: [
    { category: 'Electronics' },
    { price: { [Op.gt]: 100 } }
]
```

- **Op.or**: category equals ‘Electronics’ **or** price greater than 100

```
[Op.or]: [
    {
        category: 'Electronics',
        active: true
    },
    { price: { [Op.gt]: 100 } }
]
```

- OR operator only applies to one level of nesting at a time
- Condition: (category equals ‘Electronics’ **and** active is true) **or** price greater than 100

COPYRIGHT © 2024. SINGAPORE INSTITUTE OF TECHNOLOGY AND NANYANG POLYTECHNIC. ALL RIGHTS RESERVED.

Logical Operators

NOT

- The operator **Op.not** is used to negate the condition.

```
category: { [Op.not]: 'Clothing' }
```

- **Op.not**: category is not ‘Clothing’

```
[Op.not]: {
    category: 'Electronics',
    price: { [Op.gt]: 100 }
}
```

- **Op.not**: not (category equals ‘Electronics’ and price greater than 100)

COPYRIGHT © 2024. SINGAPORE INSTITUTE OF TECHNOLOGY AND NANYANG POLYTECHNIC. ALL RIGHTS RESERVED.

Read (find by PK)

endpoint ‘GET /product/{id}’: retrieve a product by ID

routes/product.js

```
// Define a GET endpoint to get a product by ID
router.get('/:id', async (req, res) => {
    const id = req.params.id;
    const product = await Product.findByPk(id);
    if (!product) {
        res.sendStatus(404);
        return;
    }
    res.json(product);
});
```

- The method **findByPk()** is used to retrieve a single record from a table based on its primary key value.

- Sequelize also provides another method **findOne()** to retrieve a single record with where condition.

COPYRIGHT © 2024. SINGAPORE INSTITUTE OF TECHNOLOGY AND NANYANG POLYTECHNIC. ALL RIGHTS RESERVED.

Update

endpoint ‘PUT /product/{id}’: update a product by ID

routes/product.js

```
// Define a PUT endpoint to update a product by ID
router.put('/:id', (req, res) => {
    const id = req.params.id;
    const product = await Product.findByPk(id);
    if (!product) {
        res.sendStatus(404);
        return;
    }
    // Update the product
    let data = req.body;
    await Product.update(data, { where: { id: id } });
    res.json({ message: 'Product updated' });
});
```

- The method **update()** with the where condition will find the record with the provided id and update the record with the new data.

COPYRIGHT © 2024. SINGAPORE INSTITUTE OF TECHNOLOGY AND NANYANG POLYTECHNIC. ALL RIGHTS RESERVED.

Delete

endpoint ‘DELETE /product/{id}’: delete a product by ID

routes/product.js

```
// Define a DELETE endpoint to delete a product by ID
router.delete('/:id', (req, res) => {
    const id = req.params.id;
    const product = await Product.findByPk(id);
    if (!product) {
        res.sendStatus(404);
        return;
    }
    // Delete the product
    await Product.destroy({ where: { id: id } });
    res.json({ message: 'Product deleted' });
});
```

- The method **destroy()** with the where condition will delete the record with the provided id.

COPYRIGHT © 2024. SINGAPORE INSTITUTE OF TECHNOLOGY AND NANYANG POLYTECHNIC. ALL RIGHTS RESERVED.

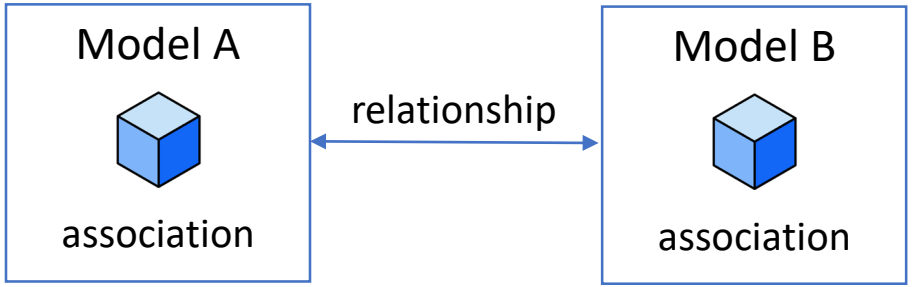
Associations

COPYRIGHT © 2024. SINGAPORE INSTITUTE OF TECHNOLOGY AND NANYANG POLYTECHNIC. ALL RIGHTS RESERVED.

Sequelize Associations

Sequelize **associations** are used to define **relationships** between tables (models) in a relational database.

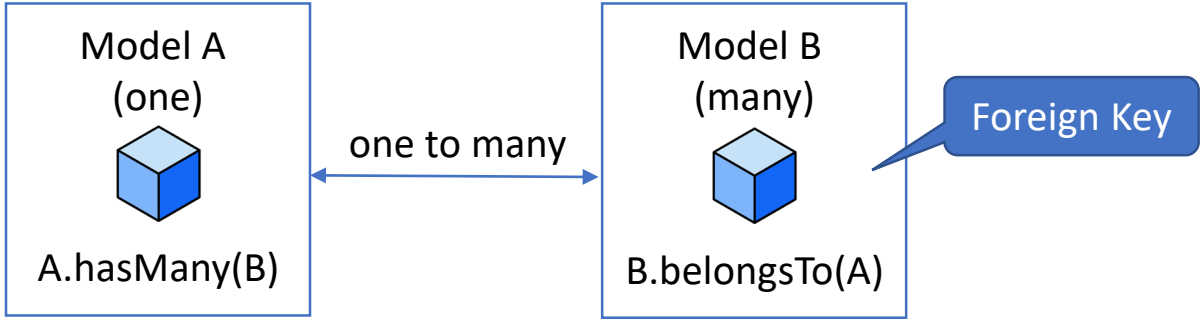
- One-to-One relationship: use hasOne and belongsTo associations
- One-to-Many relationship: use hasMany and belongsTo associations
- Many-to-Many relationship: use two belongsToMany associations



COPYRIGHT © 2024. SINGAPORE INSTITUTE OF TECHNOLOGY AND NANYANG POLYTECHNIC. ALL RIGHTS RESERVED.

One-to-Many Relationship

- The **one-to-many** relationship is a common type of database relationship in which one record in one table is associated with multiple records in another table.
- In Sequelize, you can define and work with one-to-many relationships using associations **hasMany** and **belongsTo**.
- One-to-Many relationship example:
  - One Category (Model A) has many Products (Model B)



COPYRIGHT © 2024. SINGAPORE INSTITUTE OF TECHNOLOGY AND NANYANG POLYTECHNIC. ALL RIGHTS RESERVED.

hasMany Association

- One Category has many Products

models/Catogory.js

```
Category.associate = (models) => {
    Category.hasMany(models.Product, {
        onDelete: "cascade"
    });
};
```

- The method **hasMany()** defines the association for the one-to-many relationship at the one side.
- **onDelete: “cascade”** means if a category is deleted, all its associated products are deleted as well.

COPYRIGHT © 2024. SINGAPORE INSTITUTE OF TECHNOLOGY AND NANYANG POLYTECHNIC. ALL RIGHTS RESERVED.

belongsTo Association

- One Product belongs to one Category

models/Product.js

```
Product.associate = (models) => {
    Product.belongsTo(models.Category, {
        foreignKey: "categoryId",
        as: "category"
    });
};
```

- The foreign key will be generated automatically at the many side (Product).
- The default foreign key column name in the Product table will be categoryId.

- The method **belongsTo()** defines the association for the one-to-many relationship at the many side.
- To use your custom foreign key name, you can specify it explicitly using the **foreignKey** option.
- The option **as** defines the alias, which allows you to access the associated Category model using a preferred name when querying.

COPYRIGHT © 2024. SINGAPORE INSTITUTE OF TECHNOLOGY AND NANYANG POLYTECHNIC. ALL RIGHTS RESERVED.

include Option

- Include Category when querying Product

```
// Define a GET endpoint to list all products
router.get('/', async (req, res) => {
    let minPrice = parseFloat(req.query.minPrice) || 0;
    let list = await Product.findAll({
        where: {
            price: { [Op.gte]: minPrice }
        },
        include: {
            model: Category,
            as: "category",
            attributes: ['name']
        },
        order: [['price', 'ASC']],
    });
    res.json(list);
});
```

- The **include** option is used to include related models when querying the main model. It enables eager loading of associated data in a single query.
- **model: Category** specifies the model to include
- **as: "category"** specifies the alias for the included model, same as the alias defined in the belongsTo association
- **attributes: ['name']** specifies which columns from the included model you want to retrieve
- The include option can also be used in other query methods like findByPk.

COPYRIGHT © 2024. SINGAPORE INSTITUTE OF TECHNOLOGY AND NANYANG POLYTECHNIC. ALL RIGHTS RESERVED.

Input Validation

COPYRIGHT © 2024. SINGAPORE INSTITUTE OF TECHNOLOGY AND NANYANG POLYTECHNIC. ALL RIGHTS RESERVED.

Input Validation

Web API input validation refers to the process of validating the data sent to a web API, especially in the request body.

- **Data Type** Validation: matches expected data type, e.g. string, number
- **Length** Validation: check minimum and maximum length for string
- **Range** Validation: check minimum and maximum range for number or date
- **Format** Validation: matches expected format for string, e.g. email, url, regular expression
- **Whitelist** Validation: only allows value in a whitelist

COPYRIGHT © 2024. SINGAPORE INSTITUTE OF TECHNOLOGY AND NANYANG POLYTECHNIC. ALL RIGHTS RESERVED.

Popular Libraries

- Express-validator
  - If you are building a Node.js web API using Express.js and need server-side validation and sanitization, Express-validator is a great choice.
- Yup
  - If you are working on a front-end application (e.g., a React app) and need client-side validation with reusable schemas that can be shared between the client and server, Yup is a strong option.

COPYRIGHT © 2024. SINGAPORE INSTITUTE OF TECHNOLOGY AND NANYANG POLYTECHNIC. ALL RIGHTS RESERVED.

Yup

Yup is a general-purpose JavaScript validation library suitable for both front-end and back-end.

- **Schema Definition**: You can define a validation schema that describes the structure and validation rules for the input data.
- **Chainable Methods**: You can chain validation functions together to create complex validation rules.
- **Data Shaping**: Yup not only validates data but can also transform it based on your schema.
- **Error Messages**: You can specify the custom error message for each validation rule.

```
npm i yup
```

COPYRIGHT © 2024. SINGAPORE INSTITUTE OF TECHNOLOGY AND NANYANG POLYTECHNIC. ALL RIGHTS RESERVED.

Define Schema

- Define validation schema when creating product

```
let data = req.body;
// Validate request body
let validationSchema = yup.object({
    name: yup.string().trim().min(3).max(100).required(),
    price: yup.number().min(0).required(),
});

try {
    let validData = await validationSchema.validate(data,
        { abortEarly: false });
    // TODO: Process valid data
}
catch (err) {
    res.status(400).json({ errors: err.errors });
}
```

- **name**: It should be a string, with leading and trailing whitespace trimmed, having a minimum length of 3 characters, a maximum length of 100 characters, and it is required (cannot be empty or missing).
- **price**: It should be a number, with a minimum value of 0, and it is required.

COPYRIGHT © 2024. SINGAPORE INSTITUTE OF TECHNOLOGY AND NANYANG POLYTECHNIC. ALL RIGHTS RESERVED.

Validate Input

- Validate input data after the schema is defined

```
let data = req.body;
// Validate request body
let validationSchema = yup.object({
    name: yup.string().trim().min(3).max(100).required(),
    price: yup.number().min(0).required(),
});

try {
    let validData = await validationSchema.validate(data,
        { abortEarly: false });
    // TODO: Process valid data
}
catch (err) {
    res.status(400).json({ errors: err.errors });
}
```

- The **validate()** method is used to validate data against the schema and returns the valid data and any applied transformations.
- The **abortEarly** option is set to false, which means that validation will continue even if there are multiple validation errors.
- If there are validation **errors**, the code enters the catch block. It sends a response with a status code of 400 (Bad Request) and includes the error messages.

COPYRIGHT © 2024. SINGAPORE INSTITUTE OF TECHNOLOGY AND NANYANG POLYTECHNIC. ALL RIGHTS RESERVED.

Yup Functions

Data Transformation

- **Common**: required(), oneOf(array), default(value)
- **String**: string(), trim(), lowercase(), uppercase(), length(limit), min(limit), max(limit), matches(regex), email(), url()
- **Number**: number(), min(limit), max(limit), integer()
- **Date**: date(), min(limit), max(limit)
- **Boolean**: boolean()

JSON date value example

- **Date Only**: “2020-11-20” for 20 Nov 2020
- **Date Time**: “2020-11-20T16:50:00Z” for 20 Nov 2020 16:50:00

COPYRIGHT © 2024. SINGAPORE INSTITUTE OF TECHNOLOGY AND NANYANG POLYTECHNIC. ALL RIGHTS RESERVED.

Input Sanitization

Input sanitization is the process of cleaning input data to ensure it is safe and reliable before it is processed further.

- **Security:** Prevent security vulnerabilities and protect against attacks such as SQL injection, cross-site scripting (XSS), and command injection.
- **Data Integrity:** Ensure the data meets the required quality standards for processing and storage. This can include removing whitespace, converting text to lowercase, or reformatting data for consistency.

COPYRIGHT © 2024. SINGAPORE INSTITUTE OF TECHNOLOGY AND NANYANG POLYTECHNIC. ALL RIGHTS RESERVED.

Summary

1. Describe Sequelize ORM.
2. Understand Sequelize CRUD Operations.
3. Describe Sequelize Associations.
4. Implement Input Validation using Yup.

COPYRIGHT © 2024. SINGAPORE INSTITUTE OF TECHNOLOGY AND NANYANG POLYTECHNIC. ALL RIGHTS RESERVED.

NANYANG THE INNOVATIVE POLYTECHNIC

School of Information Technology

SINGAPORE INSTITUTE OF TECHNOLOGY

COPYRIGHT © 2024. SINGAPORE INSTITUTE OF TECHNOLOGY AND NANYANG POLYTECHNIC. ALL RIGHTS RESERVED.

Credits

This learning unit was developed by:

Li Haiyun, School of Information Technology, Nanyang Polytechnic

References:

- Coding stickers created by kerismaker - Flaticon
  https://www.flaticon.com/stickers-pack/web-development-195

COPYRIGHT © 2024. SINGAPORE INSTITUTE OF TECHNOLOGY AND NANYANG POLYTECHNIC. ALL RIGHTS RESERVED.